Л1 Метод штрафов. ADMM

До этого параграфа мы рассматривали лишь безусловные задачи и задачи оптимизации на множестве. Между тем, общая постановка оптимизационной задачи (**??**) включает ограничения вида неравенств и равенств, методы для которой нами ещё не были разработаны. Этот параграф посвящён построению подходов к таким задачам, в частности — методу штрафов.

Л1.1 Штрафная функция

Рассмотрим следующую задачу с ограничениями:

$$\begin{aligned} \min_{x\in\mathbb{R}^d} \quad & f(x) \\ \text{s.t.} \quad & h_j(x) = 0, \quad j = \overline{1,m}. \end{aligned} \tag{Л1.1}$$

Возьмем некоторое $\rho > 0$ и немного модифицируем нашу задачу:

$$\begin{aligned} \min_{x\in\mathbb{R}^d} \quad & f(x) + \rho \cdot \frac{1}{2}\sum_{j=1}^{m} h_j^2(x) \\ \text{s.t.} \quad & h_j(x) = 0, \quad j = \overline{1,m}. \end{aligned}$$

Замечание Л1.1. Можно заметить, что новая задача эквивалентна старой, так как для x, удовлетворяющих ограничениям, «добавка» равна 0.

Теперь запишем безусловную задачу с такой же целевой функцией:

$$\min_{x\in\mathbb{R}^d} \left[f_\rho(x) = f(x) + \rho \cdot \frac{1}{2}\sum_{j=1}^{m} h_j^2(x) \right]. \tag{Л1.2}$$

Легко проверить, что полученная задача не эквивалентна исходной: безусловный минимум $f_\rho(x)$ вообще не обязан удовлетворять ограничениям.
Задача с ограничениями стала задачей без ограничений, такое мы уже умеем решать.
При таком переходе f_ρ называют *штрафной функцией*, а ρ — *параметром штрафа*.
Попробуем увеличить штраф за нарушение ограничений. Предельный переход по ρ:

$$\lim_{\rho\to+\infty} f_\rho(x) = \begin{cases} f(x), & h_j(x) = 0, \quad j = \overline{1,m} \\ +\infty, & \text{иначе} \end{cases}$$

Замечание Л1.2. Есть надежда, что минимизируя f_ρ при решении штрафной задачи для достаточно большого ρ, мы получим неплохое решение и для исходной задачи.

Добавим еще ограничения вида неравенств:

$$\begin{aligned} \min_{x\in\mathbb{R}^d} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = \overline{1,n} \\ & h_j(x) = 0, \quad j = \overline{1,m}. \end{aligned}$$

Как их добавить в штраф? Сделаем это с помощью положительной «срезки»:

$$f_\rho(x) = f(x) + \rho \cdot \frac{1}{2}\sum_{j=1}^{m} h_j^2(x) + \rho \cdot \frac{1}{2}\sum_{i=1}^{n} (g_i^+(x))^2,$$

где $y^+ = \max\{y, 0\}$. Штраф активируется только тогда, когда нарушено неравенство.

Л1.2 Свойства решений штрафной задачи

Утверждение Л1.1. Пусть x^* — решение исходной задачи (Л1.1), а x^*_ρ — решение соответствующей штрафной задачи (Л1.2) с $\rho > 0$, тогда

$$f(x^*) \geq f(x^*_\rho).$$

Доказательство. Воспользуемся тем, что x^* удовлетворяет ограничениям, а x^*_ρ — безусловный минимум f_ρ:

$$f(x^*) = f_\rho(x^*) \geq \min_{x \in \mathbb{R}^d} f_\rho(x) = f_\rho(x^*_\rho) \geq f(x^*_\rho).$$

■

Этот результат говорит о том, что либо $f(x^*) = f(x^*_\rho)$, либо мы нарушаем ограничения. Но за счет ρ с этим можно бороться. Следующие два свойства посвящены этому.

Утверждение Л1.2. С увеличением параметра штрафа ρ решения штрафной задачи (Л1.2), при их существовании, гарантированно не ухудшают степень нарушения ограничений, то есть для $\rho_1 > \rho_2$ следует, что

$$\sum_{j=1}^{m} h_j^2(x^*_{\rho_2}) \geq \sum_{j=1}^{m} h_j^2(x^*_{\rho_1}),$$

где $x^*_{\rho_1}$ и $x^*_{\rho_2}$ — решения соответствующих штрафных задач.

Доказательство. Пользуемся тем, что $x^*_{\rho_1}$ и $x^*_{\rho_2}$ — решения соответствующих штрафных задач:

$$f(x^*_{\rho_2}) + \rho_1 \cdot \frac{1}{2} \sum_{j=1}^{m} h_j^2(x^*_{\rho_2}) \geq f(x^*_{\rho_1}) + \rho_1 \cdot \frac{1}{2} \sum_{j=1}^{m} h_j^2(x^*_{\rho_1})$$

$$f(x^*_{\rho_1}) + \rho_2 \cdot \frac{1}{2} \sum_{j=1}^{m} h_j^2(x^*_{\rho_1}) \geq f(x^*_{\rho_2}) + \rho_2 \cdot \frac{1}{2} \sum_{j=1}^{m} h_j^2(x^*_{\rho_2})$$

Складываем и делим на $\rho_1 - \rho_2 > 0$ и получаем:

$$\sum_{j=1}^{m} h_j^2(x^*_{\rho_2}) \geq \sum_{j=1}^{m} h_j^2(x^*_{\rho_1}).$$

■

Утверждение Л1.3. Пусть функция f и функции h_j являются непрерывными для всех $j = \overline{1,m}$. Пусть X^* — множество решений исходной условной задачи оптимизации (Л1.1) и для $x^* \in X^*$ ограничено множество

$$U = \left\{ x \in \mathbb{R}^d \;\middle|\; f(x) \leq f(x^*) \right\}.$$

Тогда для любого $e > 0$ существует $\rho(e) > 0$ такое, что множество решений штрафной задачи (Л1.2) X^*_ρ для любых $\rho \geq \rho(e)$ содержится в

$$X^*_e = \left\{ x \in \mathbb{R}^d \;\middle|\; \exists x^* \in X^* : \left\| x - x^* \right\|_2 \leq e \right\}.$$

Замечание Л1.3. Ограниченность U нужна для того, чтобы гарантировать, что вне ограничений функция f ведет себя «адекватно» и штрафная функция просто не улетит в $-\infty$. По факту это и гарантирует существование и непустоту X^*_ρ.

Доказательство. Доказательство проведём от противного. Пусть существуют такие $e > 0$ и последовательность $\{\rho_i\} \to \infty$, что $X^*_{\rho_i}$ не содержатся в X^*_e, то есть существуют

$$x^*_i \in X^*_{\rho_i} \setminus X^*_e. \tag{Л1.3}$$

Согласно Утверждению Л1.1:

$$f\big(x^*\big) \geq f\big(x^*_i\big), \tag{Л1.4}$$

а значит все x^*_i лежат в ограниченном множестве U.
По теореме Больцано–Вейерштрасса (Теорема 2 Параграфа 9 Главы 1 из [1]) из ограниченной последовательности можно выделить сходящуюся подпоследовательность:

$$\tilde{x}^*_i \to \tilde{x}^*.$$

Посмотрим, что мы можем сказать про $\tilde{x}^*$. Используя непрерывность f, можем перейти к пределу в неравенстве (Л1.4):

$$f\big(x^*\big) \geq \lim_{i\to\infty} f\big(\tilde{x}^*_i\big) = f\big(\tilde{x}^*\big).$$

Покажем, что дополнительно $\tilde{x}^*$ удовлетворяет исходным ограничениям h. Пусть для какого-то $k \in \overline{1,m}$ ограничение h_k не выполняется:

$$h_k\big(\tilde{x}^*\big) \neq 0.$$

В силу непрерывности h_k, можно заметить, что, начиная с достаточно большого номера i, выполнено

$$\big|h_k\big(\tilde{x}^*_i\big)\big| \geq \frac{1}{2}\big|h_k\big(\tilde{x}^*\big)\big| > 0.$$

Но тогда в пределе $\tilde{\rho}_i \to +\infty$ для последовательности штрафных функций верно:

$$f_{\tilde{\rho}_i}\big(\tilde{x}^*_i\big) = f\big(\tilde{x}^*_i\big) + \tilde{\rho}_i \cdot \frac{1}{2}\sum_{j=1}^{m} h^2_j\big(\tilde{x}^*_i\big) \to +\infty.$$

Пришли к противоречию, так как $f_{\tilde{\rho}_i}\big(\tilde{x}^*_i\big) \leq f\big(x^*\big)$. Значит, $\tilde{x}^*$ удовлетворяет всем ограничениям, а значит, $\tilde{x}^* \in X^*$. Но тогда по определению предела последовательности, начиная с некоторого номера i, будет выполняться

$$\tilde{x}^*_i \in \Big\{\, x \in \mathbb{R}^d \;\Big|\; \big\|x - \tilde{x}^*\big\|_2 \leq e \,\Big\} \subseteq X^*_e.$$

Пришли к противоречию с (Л1.3). ■

Л1.3 Двойственный подъем и аугментация

Рассмотрим задачу с ограничениями

$$\begin{aligned} \min_{x \in \mathbb{R}^d} \quad & f\big(x\big) \\ \text{s.t.} \quad & Ax = b, \end{aligned} \tag{Л1.5}$$

где $A \in \mathbb{R}^{n \times d}$, $b \in \mathbb{R}^n$. Лагранжиан (Определение **??**) для неё записывается как:

$$\mathcal{L}(x, \lambda) = f\big(x\big) + \lambda^\top (Ax - b).$$

Зная лагранжиан, можем перейти к двойственной функции (Определение **??**):

$$g\big(\lambda\big) = \inf_{x \in \mathbb{R}^d} \mathcal{L}(x, \lambda).$$

Двойственная задача записывается следующим образом:

$$\max_{\lambda \in \mathbb{R}^n} \quad g\big(\lambda\big).$$

Тогда запустим градиентный подъем с шагом α для максимизации двойственной функции g:

$$\lambda^{k+1} = \lambda^k + \alpha \nabla_{\lambda_k} \left(\inf_{x \in \mathbb{R}^d} \big[f(x) + \lambda_k^\top (Ax - b) \big] \right).$$

Перепишем иначе:

$$\begin{aligned} x^{k+1} &= \operatorname*{argmin}_{x \in \mathbb{R}^d} \big[f\big(x\big) + \lambda_k^\top (Ax - b) \big] = \operatorname*{argmin}_{x \in \mathbb{R}^d} \mathcal{L}(x, \lambda^k) \\ \lambda^{k+1} &= \lambda^k + \alpha \nabla_{\lambda_k} \big(f\big(x^{k+1}\big) + \lambda_k^\top \big(Ax^{k+1} - b \big) \big) = \lambda^k + \alpha \big(Ax^{k+1} - b \big). \end{aligned}$$

То есть на каждой итерации считается новый x^{k+1} для λ^k и обновляется по нему λ^{k+1}. Такой метод называется *двойственный подъем*.
Перейдем к аугментации, другими словами — к добавлению штрафа. Уже знаем, что такая «добавка» не меняет задачу:

$$\begin{aligned} \min_{x \in \mathbb{R}^d} \quad & f\big(x\big) + \frac{\rho}{2} \big\| Ax - b \big\|_2^2 \\ \text{s.t.} \quad & Ax = b. \end{aligned}$$

Лагранжиан аугментированной задачи:

$$\mathcal{L}_\rho(x, \lambda) = f\big(x\big) + \lambda^\top (Ax - b) + \frac{\rho}{2} \big\| Ax - b \big\|_2^2.$$

Процедура двойственного подъема для аугментированной задачи:

$$\begin{aligned} x^{k+1} &= \operatorname*{argmin}_{x \in \mathbb{R}^d} \mathcal{L}_\rho(x, \lambda^k) \\ \lambda^{k+1} &= \lambda^k + \rho \big(Ax^{k+1} - b \big). \end{aligned}$$

Замечание Л1.4. Использование аугментации позволяет не устремлять ρ к бесконечности.

Замечание Л1.5. Шаг α специально заменен на ρ, чтобы подбирать один параметр для метода.

Теперь можно перейти к следующему методу.

Л1.4 ADMM

Пусть в задаче (Л1.5) целевая функция и ограничения распадаются на две группы переменных. Иначе говоря, пусть задача имеет вид:

$$\begin{aligned} \min_{\substack{x\in\mathbb{R}^{d_x}\\ y\in\mathbb{R}^{d_y}}} \quad & f\big(x\big) + g\big(y\big) \\ \text{s.t.} \quad & Ax + By = c, \end{aligned} \tag{Л1.6}$$

где $A \in \mathbb{R}^{n\times d_x}$, $B \in \mathbb{R}^{n\times d_y}$, $c \in \mathbb{R}^n$. Из практики мы знаем, что регуляризация часто делает задачу более стабильной, давайте добавим такую регуляризацию, что наша исходная задача не изменится. Тогда мы получим аугментированную задачу:

$$\begin{aligned} \min_{\substack{x\in\mathbb{R}^{d_x}\\ y\in\mathbb{R}^{d_y}}} \quad & f\big(x\big) + g\big(y\big) + \frac{\rho}{2}\big\|Ax + By - c\big\|_2^2 \\ \text{s.t.} \quad & Ax + By = c. \end{aligned}$$

Запишем лагранжиан аугментированной задачи:

$$\mathcal{L}_\rho(x, y, \lambda) = f\big(x\big) + g\big(y\big) + \lambda^\top\big(Ax + By - c\big) + \frac{\rho}{2}\big\|Ax + By - c\big\|_2^2.$$

Как мы уже знаем, мы хотим минимизировать лагранжиан по прямым переменным и минимизировать по двойственным. Давайте по прямым делать явную минимизацию, а по двойственным градиентный подъем. Полученный метод называется *метод альтернативных направлений*, он же Alternating Direction Method of Multipliers (ADMM):

Алгоритм Л1.1 ADMM

Вход: стартовые точки $y^0 \in \mathbb{R}^{d_y}, \lambda^0 \in \mathbb{R}^n$, параметр $\rho \in \mathbb{R}$, количество итераций K

1: **for** $k = 0, 1, \ldots, K-1$ **do**
2: $\quad x^{k+1} = \operatorname{argmin}_{x\in\mathbb{R}^{d_x}} \mathcal{L}_\rho\big(x, y^k, \lambda^k\big)$
3: $\quad y^{k+1} = \operatorname{argmin}_{y\in\mathbb{R}^{d_y}} \mathcal{L}_\rho\big(x^{k+1}, y, \lambda^k\big)$
4: $\quad \lambda^{k+1} = \lambda^k + \rho\big(Ax^{k+1} + By^{k+1} - c\big)$
5: **end for**

Выход: $\frac{1}{K}\sum_{k=1}^K x^k$, $\frac{1}{K}\sum_{k=1}^K y^k$, $\frac{1}{K}\sum_{k=1}^K \lambda^k$

Поясним смысл названия ADMM:

- Alternating Direction: минимизация по x и y происходит не одновременно, а последовательно, одна за другой.
- Multipliers: наличие двойственных множителей Лагранжа λ.

Замечание Л1.6. В доказательстве сходимости будем использовать версию алгоритма с другим порядком обновления точек:

$$y^{k+1} = \operatorname*{argmin}_{y\in\mathbb{R}^{d_y}} \mathcal{L}_\rho\big(x^k, y, \lambda^k\big) \tag{Л1.7}$$

$$\lambda^{k+1} = \lambda^k + \rho\big(Ax^k + By^{k+1} - c\big) \tag{Л1.8}$$

$$x^{k+1} = \operatorname*{argmin}_{x\in\mathbb{R}^{d_x}} \mathcal{L}_\rho\big(x, y^{k+1}, \lambda^{k+1}\big). \tag{Л1.9}$$

Теорема Л1.1. Пусть задача оптимизации (Л1.6) с выпуклыми и дружественными с точки зрения вычислений argmin функциями f и g решается с помощью ADMM (Алгоритм Л1.1 с обновлением (Л1.7)–(Л1.9)). Тогда справедлива следующая оценка сходимости:

$$\mathcal{L}_0\left(\frac{1}{K}\sum_{k=1}^{K}x^k, \frac{1}{K}\sum_{k=1}^{K}y^k, \lambda^*\right) - \mathcal{L}_0\left(x^*, y^*, \frac{1}{K}\sum_{k=1}^{K}\lambda^k\right) \leq \frac{1}{2K}\|z^* - z^0\|_P^2,$$

где $\mathcal{L}_0$ — Лагранжиан без аугментации и

$$P = \begin{pmatrix} \rho A^\top A & 0 & A^\top \\ 0 & 0 & 0 \\ A & 0 & \frac{1}{\rho}I_n \end{pmatrix}, \quad z^* = \begin{pmatrix} x^* \\ y^* \\ \lambda^* \end{pmatrix}, \quad z^0 = \begin{pmatrix} x^0 \\ y^0 \\ \lambda^0 \end{pmatrix}.$$

Замечание Л1.7. Определение P–нормы приводилось в Параграфе **??** (Определение **??**).

Доказательство. Запишем условие оптимальности для уравнения (Л1.7):

$$\nabla g\big(y^{k+1}\big) + B^\top\lambda^k + \rho B^\top\big(Ax^k + By^{k+1} - c\big) = 0. \tag{Л1.10}$$

Формула (Л1.8) обновления λ^{k+1}:

$$\lambda^{k+1} - \lambda^k = \rho\big(Ax^k + By^{k+1} - c\big). \tag{Л1.11}$$

Условие оптимальности для уравнения (Л1.9):

$$\nabla f\big(x^{k+1}\big) + A^\top\lambda^{k+1} + \rho A^\top\big(Ax^{k+1} + By^{k+1} - c\big) = 0. \tag{Л1.12}$$

Используя равенства (Л1.10), (Л1.11) и (Л1.12), выразим градиент $\mathcal{L}_0$ в точке $z^{k+1} = \big(x^{k+1}, y^{k+1}, \lambda^{k+1}\big)^\top$. Для удобства и единообразия записи, градиент по λ будем брать со знаком минус:

$$\begin{aligned}\nabla\mathcal{L}_0\big(z^{k+1}\big) &= \begin{pmatrix} \nabla_x\mathcal{L}_0\big(x^{k+1}, y^{k+1}, \lambda^{k+1}\big) \\ \nabla_y\mathcal{L}_0\big(x^{k+1}, y^{k+1}, \lambda^{k+1}\big) \\ -\nabla_\lambda\mathcal{L}_0\big(x^{k+1}, y^{k+1}, \lambda^{k+1}\big) \end{pmatrix} = \begin{pmatrix} \nabla f\big(x^{k+1}\big) + A^\top\lambda^{k+1} \\ \nabla g\big(y^{k+1}\big) + B^\top\lambda^{k+1} \\ -\big(Ax^{k+1} + By^{k+1} - c\big) \end{pmatrix} \\ &= -\begin{pmatrix} \rho A^\top\big(Ax^{k+1} + By^{k+1} - c\big) \\ -B^\top\big(\lambda^{k+1} - \lambda^k\big) + \rho B^\top\big(Ax^k + By^{k+1} - c\big) \\ \frac{1}{\rho}\big(\lambda^{k+1} - \lambda^k\big) + A\big(x^{k+1} - x^k\big) \end{pmatrix}.\end{aligned}$$

Воспользуемся уравнением (Л1.11):

$$\nabla\mathcal{L}_0\big(z^{k+1}\big) = -\begin{pmatrix} A^\top\big(\lambda^{k+1} - \lambda^k\big) + \rho A^\top A\big(x^{k+1} - x^k\big) \\ 0 \\ \frac{1}{\rho}\big(\lambda^{k+1} - \lambda^k\big) + A\big(x^{k+1} - x^k\big) \end{pmatrix}.$$

Обозначим

$$P = \begin{pmatrix} \rho A^\top A & 0 & A^\top \\ 0 & 0 & 0 \\ A & 0 & \frac{1}{\rho}I_n \end{pmatrix}.$$

Заметим, что матрица симметричная и положительно полуопределённая. Запишем:

$$\nabla \mathcal{L}_0\big(z^{k+1}\big) = -P\begin{pmatrix} x^{k+1} - x^k \\ y^{k+1} - y^k \\ \lambda^{k+1} - \lambda^k \end{pmatrix} = -P\big(z^{k+1} - z^k\big).$$

Тогда, переписывая в терминах P–нормы $\|z\|_P = \langle z, Pz\rangle$ (Определение **??**), получаем:

$$\begin{aligned}\big\langle \nabla \mathcal{L}_0\big(z^{k+1}\big), z^{k+1} - z\big\rangle &= -\big\langle P\big(z^{k+1} - z^k\big), z^{k+1} - z\big\rangle \\ &= \frac{1}{2}\Big(\big\|z^k - z\big\|_P^2 - \big\|z^{k+1} - z\big\|_P^2 - \big\|z^{k+1} - z^k\big\|_P^2\Big) \\ &\le \frac{1}{2}\Big(\big\|z^k - z\big\|_P^2 - \big\|z^{k+1} - z\big\|_P^2\Big).\end{aligned}$$

Суммируем по всем k от 0 до $K-1$, усредняем и сворачиваем телескопическую сумму справа:

$$\frac{1}{K}\sum_{k=0}^{K-1}\big\langle \nabla \mathcal{L}_0\big(z^{k+1}\big), z^{k+1} - z\big\rangle \le \frac{1}{2K}\Big(\big\|z^0 - z\big\|_P^2 - \big\|z^K - z\big\|_P^2\Big) \le \frac{\big\|z^0 - z\big\|_P^2}{2K}. \quad (\text{Л1.13})$$

Воспользуемся тем, что $\mathcal{L}_0$ выпукла по (x, y) и вогнута по λ. Тогда для любого $z = (x, y, \lambda)$ и каждого k верно

$$\mathcal{L}_0\big(x^{k+1}, y^{k+1}, \lambda\big) - \mathcal{L}_0\big(x, y, \lambda^{k+1}\big) \le \big\langle \nabla \mathcal{L}_0\big(z^{k+1}\big), z^{k+1} - z\big\rangle. \quad (\text{Л1.14})$$

Применим неравенство Йенсена (**??**) к выпуклой и вогнутой частям, получим:

$$\mathcal{L}_0\left(\frac{1}{K}\sum_{k=1}^K x^k, \frac{1}{K}\sum_{k=1}^K y^k, \lambda\right) - \mathcal{L}_0\left(x, y, \frac{1}{K}\sum_{k=1}^K \lambda^k\right) \le \frac{1}{K}\sum_{k=1}^K \big(\mathcal{L}_0\big(z^k\big) - \mathcal{L}_0(z)\big).$$

Совмещаем с оценкой через скалярное произведение (Неравенство (Л1.14)):

$$\mathcal{L}_0\left(\frac{1}{K}\sum_{k=1}^K x^k, \frac{1}{K}\sum_{k=1}^K y^k, \lambda\right) - \mathcal{L}_0\left(x, y, \frac{1}{K}\sum_{k=1}^K \lambda^k\right) \le \frac{1}{K}\sum_{k=1}^K \big\langle \nabla \mathcal{L}_0\big(z^k\big), z^k - z\big\rangle.$$

Остается подставить более раннюю оценку (Л1.13):

$$\mathcal{L}_0\left(\frac{1}{K}\sum_{k=1}^K x^k, \frac{1}{K}\sum_{k=1}^K y^k, \lambda\right) - \mathcal{L}_0\left(x, y, \frac{1}{K}\sum_{k=1}^K \lambda^k\right) \le \frac{\big\|z^0 - z\big\|_P^2}{2K}.$$

Возьмём $x = x^*$, $y = y^*$, $\lambda = \lambda^*$:

$$\mathcal{L}_0\left(\frac{1}{K}\sum_{k=1}^K x^k, \frac{1}{K}\sum_{k=1}^K y^k, \lambda^*\right) - \mathcal{L}_0\left(x^*, y^*, \frac{1}{K}\sum_{k=1}^K \lambda^k\right) \le \frac{\big\|z^0 - z^*\big\|_P^2}{2K}.$$

■

ADMM является одним из ключевых и популярных методов оптимизации. Реализован во многих солверах и часто используется как метод по умолчанию. Нестандартная формулировка самой задачи, для которой придуман ADMM, оказывается, вбирает в себя много важных частных случаев. «Непривычная» переменная y часто играет роль вспомогательной переменной. Разберем несколько прикладных примеров.

Пример Л1.1. В линейной регрессии мы ищем коэффициенты $x \in \mathbb{R}^d$, которые моделируют целевой вектор $b \in \mathbb{R}^n$ по матрице признаков $A \in \mathbb{R}^{n \times d}$. Обычный метод наименьших квадратов минимизирует квадрат ошибки $\frac{1}{2}\|Ax - b\|_2^2$. Однако во многих задачах признаков часто больше, чем наблюдений ($d > n$). В этом режиме метод наименьших квадратов сильно переобучается. Чтобы предотвратить переобучение и выполнить автоматический отбор признаков (занулить часть весов), мы хотим получить модель с высокой степенью разреженности коэффициентов. Для получения разреженности мы добавляем к функции потерь ℓ_1–регуляризацию. Эта формулировка называется *Lasso*:

$$\min_{x \in \mathbb{R}^d} \frac{1}{2}\|Ax - b\|_2^2 + \tau\|x\|_1,$$

где $\tau > 0$ определяет степень разреженности. Однако, в этой задаче нет двух переменных и ограничений, которые нужны для применения ADMM. Добавим их искусственно — введем вспомогательную переменную $y \in \mathbb{R}^d$, которая будет отвечать за негладкий ℓ_1–штраф, а чтобы не изменить оригинальную задачу, добавим ограничение на равенство x и y:

$$\begin{aligned} \min_{x,y \in \mathbb{R}^d} \quad & \frac{1}{2}\|Ax - b\|_2^2 + \tau\|y\|_1 \\ \text{s.t.} \quad & x = y. \end{aligned}$$

К этой задаче ADMM уже применим. Чтобы его записать, выпишем лагранжиан аугментированной задачи:

$$\mathcal{L}_\rho(x, y, \lambda) = \frac{1}{2}\|Ax - b\|_2^2 + \tau\|y\|_1 + \frac{\rho}{2}\|x - y + \lambda\|_2^2.$$

Получим итерации ADMM:

- **Обновление x.** Мы фиксируем y и λ и минимизируем $\mathcal{L}_\rho$ по x:

  $$\min_{x \in \mathbb{R}^d} \frac{1}{2}\|Ax - b\|_2^2 + \frac{\rho}{2}\|x - y^k + \lambda^k\|_2^2.$$

  Это безусловная сильно выпуклая квадратичная задача. Возьмем градиент по x и приравняем к нулю:

  $$\nabla_x = A^\top(Ax - b) + \rho\big(x - y^k + \lambda^k\big) = 0.$$

  Раскроем скобки и сгруппируем члены с x:

  $$A^\top Ax + \rho I_d x = A^\top b + \rho(y^k - \lambda^k).$$

  В итоге мы решаем систему линейных уравнений:

  $$(A^\top A + \rho I_d)x^{k+1} = A^\top b + \rho(y^k - \lambda^k).$$

- **Обновление y.** Фиксируя x и λ, минимизируем по y:

  $$\min_{y \in \mathbb{R}^d} \tau\|y\|_1 + \frac{\rho}{2}\|x^{k+1} - y + \lambda^k\|_2^2.$$

Поскольку $\|\cdot\|_1$ и $\|\cdot\|_2^2$ можно разложить по компонентам вектора, задача решается независимо для каждого элемента y_i. Так как функция недифференцируема, мы используем субградиент:

$$0 \in \tau\partial|y_i| - \rho\big(x_i^{k+1} - y_i + \lambda_i^k\big).$$

Аналитическим решением этого уравнения будет:

$$y^{k+1} = \operatorname{sign}\left(x^{k+1} + \lambda^k\right) \max\left\{\left|x^{k+1} + \lambda^k\right| - \frac{\tau}{\rho}, 0\right\}.$$

Этот шаг принудительно зануляет коэффициенты.

- **Обновление λ.** Обновление λ — градиентный подъем:

$$\lambda^{k+1} = \lambda^k + \rho\big(x^{k+1} - y^{k+1}\big).$$

Пример Л1.2. Восстановление матриц — это задача заполнения пропущенных пикселей. Пусть M — истинное изображение, но мы наблюдаем лишь часть пикселей — Ω. Определим оператор проекции $\mathcal{P}_\Omega(X)$, который оставляет пиксели из Ω без изменений, а остальные зануляет. Тогда наша задача найти такое изображение X, что $\mathcal{P}_\Omega(X) = \mathcal{P}_\Omega(M)$. Однако, эта задача имеет огромное количество решений, которые не будут похожи на реальные изображения. Естественные изображения сильно структурированы, а значит, такую матрицу можно хорошо приблизить матрицей малого ранга. Поиск матрицы минимального ранга это комбинаторная задача. Поэтому мы используем ядерную норму $\|X\|_*$ (сумму сингулярных чисел матрицы), которая является выпуклой релаксацией функции ранга.

$$\begin{aligned} \min_{X \in \mathbb{R}^{n \times d}} \quad & \|X\|_* \\ \text{s.t.} \quad & \mathcal{P}_\Omega(X) = \mathcal{P}_\Omega(M). \end{aligned}$$

Данная задача также не подходит для ADMM, её нужно переформулировать. Перенесем жёсткое ограничение на данные в функцию–индикатор $I_\Omega(X)$, которая равна 0, если известная часть данных совпадает с M, и $+\infty$ в противном случае, и введём вспомогательную переменную Y, которая будет отвечать за выполнение этого ограничения:

$$\begin{aligned} \min_{X, Y \in \mathbb{R}^{n \times d}} \quad & \|X\|_* + I_\Omega(Y) \\ \text{s.t.} \quad & X = Y. \end{aligned}$$

Выпишем лагранжиан аугментированной задачи:

$$\mathcal{L}_\rho(X, Y, \Lambda) = \|X\|_* + I_\Omega(Y) + \frac{\rho}{2}\|X - Y + \Lambda\|_F^2.$$

Теперь мы готовы получить итерации ADMM:

- **Обновление X.** Для этого нам нужно решить следующую задачу:

$$\min_{X \in \mathbb{R}^{n \times d}} \|X\|_* + \frac{\rho}{2}\|X - Y^k + \Lambda^k\|_F^2.$$

Найдем сингулярное разложение матрицы сдвига. Пусть $Y^k - \Lambda^k = U\Sigma V^\top$. Аналитическим решением является:

$$X^{k+1} = U \operatorname{sign}(\Sigma) \max\left\{|\Sigma| - \frac{1}{\rho}, 0\right\} V^\top.$$

- **Обновление** Y. Для этого нам нужно решить следующую задачу:

$$\min_{Y \in \mathbb{R}^{n \times d}} I_\Omega(Y) + \frac{\rho}{2}\big\|X^{k+1} - Y + \Lambda^k\big\|_F^2.$$

Здесь мы ищем матрицу Y, максимально близкую к $V = X^{k+1} + \Lambda^k$ в смысле евклидова расстояния, но при этом строго удовлетворяющую индикатору I_Ω. Это поточечная проекция: пиксели из Ω обязаны быть равны изначальным данным, а пиксели не из Ω индикатор не ограничивает, поэтому они просто принимают значения из V:

$$Y_{i,j}^{k+1} = \begin{cases} M_{i,j} & (i,j) \in \Omega; \\ X_{i,j}^{k+1} + \Lambda_{i,j}^k & (i,j) \notin \Omega. \end{cases}$$

- **Обновление** Λ**.**

$$\Lambda^{k+1} = \Lambda^k + \rho\big(X^{k+1} - Y^{k+1}\big).$$

Пример Л1.3. Современные нейронные сети имеют миллиарды параметров, каждый из которых описывается числом с плавающей точкой. Например, если модель имеет 32 миллиарда параметров и хранит веса в формате fp32 (32 бита на параметр), то для её хранения потребуется 128 гигабайт, что довольно много. Для уменьшения веса модели часто используется процедура квантизации, при которой 32–битные числа с плавающей точкой переводятся в более низкую разрядность (например, бинарные веса $\{-1, 1\}$ или 8–битные целые числа). Разберемся, как обучать такие модели. Пусть $\mathcal{Q}$ набор разрешённых дискретных значений. Тогда задачу обучения квантизованной модели можно записать как следующую задачу оптимизации:

$$\begin{aligned} \min_{X \in \mathbb{R}^{n \times d}} \quad & f(X) \\ \text{s.t.} \quad & X_i \in \mathcal{Q}, \quad i = \overline{1, n}. \end{aligned}$$

Поскольку множество $\mathcal{Q}$ дискретно, то решать данную задачу градиентными методами не получится. Однако при использовании ADMM мы можем обучать веса X непрерывным образом, а дискретизацию перенести на переменную Y. Воспользуемся уже разобранным способом свести задачу к виду, пригодному для ADMM:

$$\begin{aligned} \min_{X, Y \in \mathbb{R}^{n \times d}} \quad & f(X) + I_{\mathcal{Q}}(Y) \\ \text{s.t.} \quad & X = Y. \end{aligned}$$

Выпишем лагранжиан аугментированной задачи:

$$\mathcal{L}_\rho(X, Y, \Lambda) = f(X) + I_{\mathcal{Q}}(Y) + \frac{\rho}{2}\big\|X - Y + \Lambda\big\|_F^2$$

И получим итерации ADMM:

- **Обновление** X**.**

$$\min_{X \in \mathbb{R}^{n \times d}} f(X) + \frac{\rho}{2}\big\|X - Y^k + \Lambda^k\big\|_F^2.$$

Если раньше мы могли решить эту задачу аналитически, то сейчас $f(X)$ это сложная функция, поэтому мы будем делать несколько шагов любым методом оптимизации вместо точного решения. Важно, что в этом случае пропадают теоретические гарантии сходимости.

- **Обновление Y.**

$$\min_{Y \in \mathbb{R}^{n \times d}} I_{\mathcal{Q}}(Y) + \frac{\rho}{2}\left\|X^{k+1} - Y + \Lambda^k\right\|_F^2.$$

Нужно найти Y, элементы которого принадлежат строго $\mathcal{Q}$, чтобы он был максимально близок к $V = X^{k+1} + \Lambda^k$. Это решается поэлементным округлением. Например, для бинарной сети $\mathcal{Q} = \{-1, +1\}$ обновление имеет строгий аналитический вид:

$$Y^{k+1} = \text{sign}(V).$$

Для 8–битной квантизации мы просто округляем элементы V до ближайшего разрешённого значения.

- **Обновление Λ.**

$$\Lambda^{k+1} = \Lambda^k + \rho\big(X^{k+1} - Y^{k+1}\big).$$

Как можно было заметить в примерах, ADMM часто используется в похожих задачах по похожим принципам. Разберем основные паттерны его применения:

1. **Гладкая + Негладкая функции**. Используется в задачах минимизации хорошей функции $f(x)$ с негладким штрафом $g(x)$ (ℓ_1–норма, ядерная норма). Этот паттерн соответствует следующей форме ADMM:

$$\begin{aligned} \min_{x,y \in \mathbb{R}^d} \quad & f(x) + g(y) \\ \text{s.t.} \quad & x = y. \end{aligned}$$

Данный паттерн хорошо работает, поскольку гладкая функция уходит в x–шаг (обычно линейная система), а негладкая уходит в y–шаг, который превращается в проксимальный оператор от $g(\cdot)$ (часто имеющий красивое точное решение, вроде мягкого порога).

2. **Непрерывная функция + Жесткое ограничение**. Используется в задачах минимизации функции $f(x)$ внутри строго ограниченного (или дискретного) множества $\mathcal{C}$. ADMM для данного паттерна выглядит следующим образом:

$$\begin{aligned} \min_{x,y \in \mathbb{R}^d} \quad & f(x) + I_{\mathcal{C}}(y) \\ \text{s.t.} \quad & x = y. \end{aligned}$$

Данный паттерн хорошо работает, поскольку обновление x происходит без оглядки на границы, а y становится чистой евклидовой проекцией на множество $\mathcal{C}$ (например, округление).

3. **«Запутанный оператор»**. Применяется в задачах вида $\min_x f(x) + g(Kx)$, где матрица K «перепутывает» переменные, из-за чего g нельзя вычислить поэлементно. В данном случае ADMM применяется в следующем виде:

$$\begin{aligned} \min_{x,y} \quad & f(x) + g(y) \\ \text{s.t.} \quad & Kx = y. \end{aligned}$$

Это хорошо работает, поскольку в шаге обновления y переменные развязаны, что позволяет использовать поэлементные операции.

4. **Распределённые вычисления.** Пусть в задаче обучения данные распределены на N машинах и нужно обучить общую модель: $\min_x \sum_{i=1}^N f_i(x)$. Наивное решение данной задачи приводит к тому, что на каждой итерации машины должны усреднять локально посчитанные градиенты. Однако, можно применить ADMM: узлу i даем локальную копию x_i, и заставляем всех согласиться с центральным сервером y:

$$\begin{aligned} \min_{x_i \in \mathbb{R}^d} \quad & \sum_{i=1}^N f_i(x_i) \\ \text{s.t.} \quad & x_i = y, \quad i = \overline{1, N}. \end{aligned}$$

   Такой подход позволяет шагам по x_i вычисляться на узлах параллельно и абсолютно независимо. В то время как шаг по y на центральном сервере просто усредняет присланные локальные модели.

Отметим, что несмотря на то, что эти паттерны являются популярными, применение ADMM не ограничивается только ими, и полезно уметь видеть ADMM.
Поскольку ADMM является крайне практическим алгоритмом, приведем несколько советов по его использованию:

1. Скорость работы ADMM завязана на то, что внутренние задачи для всех переменных решаются аналитически. Если задачи не имеют аналитического решения, то их можно решать численно (например, методами второго порядка для более быстрого нахождения минимума). Однако применение итеративных методов для внутренних задач существенно увеличивает время работы ADMM. Практичным решением в случае отсутствия аналитического решения у внутренних задач является использование нескольких итераций метода оптимизации. В этом случае теряются теоретические гарантии сходимости, но на практике такой вариант зачастую работает.

2. Часто в итерациях ADMM требуется решение системы линейных уравнений вида:

$$(A^\top A + \rho I_d)x = c.$$

   Наивное решение на каждой итерации стоит $\mathcal{O}(n^3)$ операций. Однако матрица $(A^\top A + \rho I_d)$ не меняется между итерациями, что позволяет вычислить разложение Холецкого матрицы ровно один раз до начала цикла ADMM и кэшировать его. Внутри цикла поиск x будет иметь сложность $\mathcal{O}(n^2)$.

3. Параметр ρ отвечает за силу штрафа за неудовлетворение условий. Если он слишком мал, переменные будут сходиться вечность. Если он слишком большой — они «слипнутся» сразу, замедлив поиск минимума. Важным моментом является то, что при изменении ρ нужно пересчитать закэшированные значения, зависящие от этого параметра.

4. На практике ADMM быстро сходится до низкой точности, после чего долго «ползет» до высокой. Поэтому не следует использовать ADMM там, где нужна хирургическая точность на уровне машинного нуля (там лучше методы внутренней точки). В задачах, где вас устраивает более грубое решение, ADMM является хорошим выбором.

Список литературы

[1] Г. Е. Иванов. *Лекции по математическому анализу. Часть 1.* МФТИ, Москва, 2011.